

Security Test Strategy

Security Test Strategy

Revision: 1 Last modified: 2026-06-26T12:00:00Z

Master technical specification — Volume 8 (Testing & QA), nano-detail document **security**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens 10-testing-acceptance-and-qa.md §5.7 (SEC) into an implementation-ready security-test bank that traces **every test back to a threat** in [`./v05-security/threat-model.md`] §11 (test-and-validation mapping) and **every threat forward to a captured-evidence proof** the mitigation is *live, not promised*. SPEC-ONLY: it describes the harness, fixtures, evidence, gate, and the paired §1.1 mutations; it does not build the product. Sources cited inline by id — [OVERVIEW] = doc 10; [TM] = the threat model; [04_SEC] = doc 04 security spine (invariants S1–S11); [svc-policy]/[svc-pki] = Volume-3 nano-docs; [01-DP] = doc 01 data plane. Claims not grounded in the evidence base are marked **UNVERIFIED** per constitution §11.4.6 — never fabricated.

Table of contents

- 0. Scope on HelixVPN surfaces
 - 1. The threat→test trace (the spine of this bank)
 - 2. The five sub-disciplines
 - 3. Harness
 - 4. Fixtures — real attacker, real victim (§11.4.27)
 - 5. Captured evidence (§11.4.69 / .107) + self-validated analyzers
 - 6. Determinism (§11.4.50)
 - 7. Acceptance gate
 - 8. The paired §1.1 mutations (one per invariant)
 - 9. Test skeletons
 - Sources verified
-

0. Scope on HelixVPN surfaces

Security is the test type where a green summary line is most dangerous: a VPN that "connects" but leaks one DNS query deanonymises the user (overview §0). This bank covers the eleven security invariants S1–S11 [04_SEC §0.1] across every trust

boundary TB-1..TB-7 [TM §3], grouped into five sub-disciplines: **authn/authz** (S4, P6, RBAC), **injection/taint** (RLS, policy compiler, schema-lint), **secret-leak audit** (§11.4.10/.10.A), **transport/crypto** (S6/S7 DPI-evasion, S10 PQ, kill-switch S9), and **dependency/CVE**. Each test's PASS is captured evidence the *attack was attempted and blocked* — a unit assertion that the mitigation "exists" is necessary, never sufficient ([TM §11 anti-bluff floor]).

Invariant	Property	Owning surface	Bluff class guarded
S1 / I6	default-deny; fail-closed compiler	edge verdict map + policy compiler	B1 config-only
S2	device keys never leave the FFI / OS keystore	helix-core	B2 absence-of-error
S3	need-to-know map filtering before the wire	coordinator	B1
S4	two-channel auth (mTLS control / WG data)	control plane + edge	B1
S5	sub-second revocation	pki + coordinator + edge	B4 stale-state
S6 / I5	no durable connection/traffic log	DB schema + edge	B2
S7	actor-bound control-action audit	control plane	—
S8	edge hardening (rootless, seccomp, no-SSH)	edge container	—
S9	kill-switch + DNS-leak state machine	helix-core	B2
S10	hybrid PQ PSK (never PQ-only)	helix-wg handshake	B3 wrong-plane
S11	CA key in KMS/HSM, never in process	pki	—

1. The threat→test trace (the spine of this bank)

The threat model's §11 table is the **source of truth** for which test must exist; this document gives each its harness, evidence, and mutation. Every high-severity threat **MUST** have a row here; a threat with no test row is a release blocker ([TM §0]).

Threat (TM)	Invariant	SEC test id	Captured-evidence proof
T-EDGE-E-1, T-CONN-E-1	S1/I6	SEC-DEFAULT-DENY	negative E2E pcap: SYN out, zero SYN-ACK; edge verdict drop++
T-CLI-I-1, T-PKI-S-1	S2	SEC-KEY-NEVER-LEAVES	FFI boundary scan + log scan: zero private-key bytes cross FFI / hit any log
	S3		

Threat (TM)	Invariant	SEC test id	Captured-evidence proof
T-COORD-I-1, T-CONN-I-1		SEC-NEED-TO-KNOW	served-map byte-compare: device granted conn-A receives only conn-A
T-CP-S-1, T- CLI-S-1, T- COORD-S-1	S4	SEC-MTLS	handshake pcap shows client cert; expired/revoked cert ⇒ stream refused
T-PKI-E-1, R- RACE	S5	SEC-REVOKE-SUBSEC	revoke→edge-peer-removed timing CSV, p99 < 1 s
T-CP-T-2, LP- NC-1	S6	SEC-NO-LOG-SCHEMA	schemalint PASS; a CREATE TABLE flows(...) migration ⇒ build FAIL
T-CP-R-1, T- PKI-R-1	S7	SEC-AUDIT-ACTOR	every state change → audit_events with actor binding
T-EDGE-T-1	S8	SEC-EDGE-SECCOMP	execve/mount/ptrace inside edge ⇒ seccomp EPERM
T-CLI-T-1, T- DP-D-1	S9	SEC-KILLSWITCH	host pcap during drop: zero non-loopback egress, zero :53
T-DP-I-3 (fingerprint)	S6/S7 wire	SEC-DPI-EVASION	tshark classifies MASQUE as HTTP/3, no WG signature, SNI = decoy
T-DP-I-2	S10	SEC-PQ-HYBRID	handshake capture carries ML-KEM-derived PSK; non-PQ peer falls back classically
T-CP-I-1, T- CP-E-1	P6	SEC-RLS	tenant-A session cannot read tenant-B rows even with crafted WHERE
T-PKI-I-1	S11	SEC-CA-IN-KMS	pki process memory scan + config: no raw CA key; signing grant only
(supply chain)	S8	SEC-DEP-CVE	cargo audit / govulncheck / image scan: zero known-exploitable CVEs
(secrets)	§11.4.10	SEC-SECRET-LEAK	git ls-files xargs grep + git log -S empty for every secret

2. The five sub-disciplines

(A) authn/authz. SEC-MTLS proves the two-channel separation (S4): the control channel demands a CA-signed leaf cert (≤ 24 h, [04_SEC §4.3]); the data channel demands the registered WG static key (Noise IK). Negative cases: an expired leaf $\Rightarrow$ stream refused; a WG init with an unregistered key $\Rightarrow$ no handshake. SEC-RLS proves authz-below-RBAC: under `FORCE ROW LEVEL SECURITY` as the non-superuser `helix_app` role (P6), a tenant-A session reading `SELECT * FROM devices` returns zero tenant-B rows even with a crafted `WHERE tenant_id = 'B'`. SEC-RBAC: a member-role agent cannot activate a policy (`requireRole("admin", "operator"), [svc-policy §9]`).

(B) injection/taint. SEC-NO-LOG-SCHEMA is the canonical injection test inverted: a paired mutation *injects* a `CREATE TABLE flows(src_ip, dst_ip, bytes, ts)` migration and asserts the CI `schemalint` **FAILs the build** (S6, [04_SEC §6.2]). Policy-spec injection: a crafted ACL with a malformed/overlong rule must be *rejected* by the fail-closed dry-run compiler (P4), never silently widened to allow-all (T-CP-T-2). SQL-injection on the REST surface: parameterized queries only; a fuzzed tenant-id input cannot escape RLS.

(C) secret-leak audit (§11.4.10 / .10.A). Before any credential is stored, and on every pre-push, the audit runs `git ls-files | xargs grep -l <value> (tree-leak) + git log -S<value> --all --source --remotes (history-leak)` for every secret class (enroll tokens, KMS creds, leaf keys, `.env` values). Findings open a §6 incident and redact in-place ([04_SEC], §11.4.10.A). The pre-push hook's credential-pattern grep catches the escaped class in the same commit.

(D) transport/crypto. SEC-DPI-EVASION (S6/S7 wire): a passive observer (`tshark`) on the path must classify the MASQUE flow as generic HTTP/3 with **no** WG signature and a decoy SNI ([TM T-DP-I-3], `transport-masque-quick` doc). SEC-PQ-HYBRID (S10): the WG handshake carries an ML-KEM-768-derived PSK; a non-PQ peer falls back to classical (still secure today) — never PQ-only. SEC-KILLSWITCH (S9): on a forced tunnel drop, zero plaintext and zero `:53` egress the host physical interface during `Reconnecting/Blocked` (overview §5.7, defeats B2).

(E) dependency/CVE. SEC-DEP-CVE runs `cargo audit` (Rust core/edge), `govulncheck` (Go control plane), `dart pub/osv-scanner` (Flutter), and a rootless image scan on the published edge image; a known-exploitable CVE on a reachable path is a release blocker. This composes the security submodule's tooling (overview §5.7 row).

3. Harness

Sub-discipline	Harness
authn/authz (S4, P6, RBAC)	netns + pcap (mTLS handshake capture); Postgres-via-containers RLS rowset capture (overview §5.2)
injection/taint (S6, compiler)	<code>schemalint</code> against the live DB; <code>cargo test</code> compiler property tests (fail-closed)
secret-leak (§11.4.10)	<code>scripts/secret_audit.sh</code> (<code>git ls-files/git log -S</code>); pre-push hook <code>grep</code>

Sub-discipline	Harness
transport/crypto (S6/S7/S9/S10)	netns + tshark classifier; security submodule DPI/leak tooling; host-iface pcap for kill-switch
dependency/CVE	cargo audit + govulncheck + osv-scanner + rootless image scan

The whole bank runs under `make sec` (overview §9) and is dispatched by `helix_qa` as the §11.4.165 independent-verification agent for the security layer. The RLS + secret-audit subset runs on every **pre-push** (overview §9); the full bank runs in `make qa`. Container infra is rootless Podman (§11.4.161); the only sudo is netns creation (`CAP_NET_ADMIN`, scoped exception).

4. Fixtures — real attacker, real victim (§11.4.27)

Security is below the unit layer: **no mocks**. The fixtures are a real attacker and a real victim.

Fixture	What	Why real
<code>attacker_unregistered.key</code>	a WG static key never enrolled	proves the edge rejects a <i>real</i> unregistered peer (S4 data channel)
<code>expired_leaf.pem</code> / <code>revoked_leaf.pem</code>	a real leaf cert past TTL / on the revocation list	proves the control channel refuses a <i>real</i> bad cert (S4/S5)
<code>tenant_A.session</code> / <code>tenant_B.seed</code>	two real RLS-scoped DB sessions + seeded rows	proves cross-tenant denial on the <i>real</i> DB, not a stub (P6)
<code>killswitch_clean.pcap</code> / <code>killswitch_leaked_dns.pcap</code>	golden-good + golden-bad capture pair	self-validates the leak detector (§5, §11.4.107(10))
<code>masque_flow.pcap</code> / <code>wg_signature.pcap</code>	a real MASQUE flow + a real plain-WG flow	proves the DPI classifier distinguishes them (S6/S7)
<code>flows_injection.sql</code>	the forbidden <code>CREATE TABLE flows(...)</code> migration	the mutation that schema-lint MUST reject (S6)

The golden-bad fixtures are load-bearing: they are how the analyzers self-validate (§5) — an analyzer that passes its golden-bad fixture is itself the bluff (B5).

5. Captured evidence (§11.4.69 / .107) + self-validated analyzers

Every PASS cites an artifact under `qa-results/sec/<run-id>/`, scored by the challenges engine. Crucially, every security **analyzer** (leak detector, DPI classifier, FFI key-scanner, RLS rowset comparator, PQ-PSK detector) is **self-validated** against a golden-good + golden-bad fixture pair and wired into the meta-test sweep (§11.4.107(10), overview §3.3) — defeating B5.

```
// tests/analyzers/leak_detector_selfvalidation.rs
 (§11.4.107(10), overview §3.3)
#[test] fn golden_good_clean_drop_passes() {
    assert_eq!(LeakDetector::new().scan_pcap("fixtures/
        killswitch_clean.pcap").verdict, Verdict::Pass);
}
#[test] fn golden_bad_leaked_dns_fails() { // a
    seeded :53 leak MUST FAIL the analyzer
    let v = LeakDetector::new().scan_pcap("fixtures/
        killswitch_leaked_dns.pcap");
    assert_eq!(v.verdict, Verdict::Fail);
    assert!(v.findings.iter().any(|f| f.proto == "dns" && f.dport
        == 53));
}
```

Test	Artifact	§11.4.69 evidence shape
SEC-DEFAULT-DENY	deny.pcap	SYN out, zero SYN-ACK; edge drop++ counter
SEC-KILLSWITCH	killswitch.pcap	zero non-loopback + zero :53 after drop
SEC-KEY-NEVER-LEAVES	ffi_scan.json + log_scan.json	zero private-key bytes across FFI / in logs
SEC-RLS	rls_rowset.json	tenant-A rowset excludes tenant-B ids
SEC-DPI-EVASION	tshark_classify.json	proto==http3, no WG signature, SNI=decoy
SEC-PQ-HYBRID	handshake.pcap	ML-KEM PSK present; classical fallback observed
SEC-NO-LOG-SCHEMA	schemalint.log	PASS clean; mutation ⇒ FAIL
SEC-SECRET-LEAK	secret_audit.log	empty grep + empty git log -S
SEC-DEP-CVE	cargo_audit.json + govulncheck.json	zero exploitable-on-reachable-path

6. Determinism (§11.4.50)

Security verdicts are binary and MUST be reproducible: `ab_run_n_times "sec-killswitch" 3 run_killswitch` runs $N=3$ against the same artifact MD5 + same rig; the evidence-hash is over the verdict tuple (`leak_bytes: 0`, `dns_53: 0`, `fsm_state: Blocked`) and all three MUST match. A revoke-latency test that passes $p99 < 1$ s in 2 of 3 runs is **auto-FAIL** (no flake escape). The secret-leak audit is deterministic by construction (grep over a fixed tree/history). Cycle-validation runs $N=10$ for the irreversible-security floor (kill-switch, default-deny, RLS, revoke) which is also the §11.4.132 risk-ordered head (runs first, before any convenience test).

7. Acceptance gate

AC/Gate	Bar	Evidence	Phase
AC3 / SEC-DEFAULT-DENY	unauthorized host DENIED (no SYN-ACK)	<code>deny.pcap</code>	MVP (release-blocking)
AC7 / SEC-KILLSWITCH	zero plaintext + zero :53 on drop	<code>killswitch.pcap</code>	MVP (release-blocking)
AC4 / SEC-DPI-EVASION	MASQUE classified HTTP/3, no WG sig	<code>tshark_classify.json</code>	MVP
AC6 / SEC-REVOKE-SUBSEC	revoke→edge enforcement $p99 < 1$ s	revoke timing CSV	MVP
AC8 / SEC-NO-LOG-SCHEMA	schemalint PASS, mutation FAIL	<code>schemalint.log</code>	MVP
S2 / SEC-KEY-NEVER-LEAVES	zero key bytes across FFI / logs	<code>ffi_scan.json</code>	MVP
S9 (RLS) / SEC-RLS	cross-tenant rowset isolation	<code>rls_rowset.json</code>	MVP
SEC-PQ-HYBRID	ML-KEM PSK + classical fallback	<code>handshake.pcap</code>	Phase 2 (S10)
SEC-DEP-CVE	zero exploitable CVE on reachable path	scanner JSON	MVP (every build)
SEC-SECRET-LEAK	empty tree + history grep	<code>secret_audit.log</code>	MVP (every pre-push + pre-store)

A phase ships only when every required SEC cell is AUTONOMOUS_VERIFIED (overview §6 release rule); the irreversible-security floor runs first (§11.4.132). A security FAIL is never demoted to a lower severity without same-conditions evidence (§11.4.7).

8. The paired §1.1 mutations (one per invariant)

Each invariant's gate has a paired mutation that, when applied, makes the gate FAIL — proving the gate is not a bluff. The working tree is verified quiescent (§11.4.84) and restored before commit.

Gate	Mutation	Expected FAIL
CM-DEFAULT-DENY	widen edge verdict-map default to ACCEPT	SEC-DEFAULT-DENY sees a SYN-ACK on the unauthorized host
CM-LEAK-DETECTOR-SELFVALIDATED	strip the dport == 53 clause from the leak detector	golden-bad pcap now PASSES the leak → meta-test FAILs (defeats B5)
CM-NO-LOG-SCHEMA	remove the flows-shaped-table check from schemalint	the injected flows migration passes → SEC-NO-LOG-SCHEMA FAILs
CM-RLS-ENFORCED	drop FORCE ROW LEVEL SECURITY / use a superuser role	tenant-A rowset now includes tenant-B → SEC-RLS FAILs
CM-REVOKE-SUBSEC	remove the revoked-serial check at the edge	a revoked peer still reaches the LAN → SEC-REVOKE FAILs
CM-KEY-NEVER-LEAVES	expose a raw-key FFI getter	the FFI scan finds key bytes crossing the boundary → SEC-KEY FAILs
CM-DPI-EVASION	emit a plain-WG fingerprint instead of MASQUE framing	tshark classifies WG signature → SEC-DPI-EVASION FAILs

9. Test skeletons

```
# rig/killswitch_drop.sh – S4+S5+S9, the anti-bluff kill-switch
# test (defeats B2), overview §5.7
set -euo pipefail
out="qa-results/sec/$(date +%s)"; mkdir -p "$out"; pcap="$out/killswitch.pcap"
trap 'rig/netns_down.sh' EXIT #
§11.4.14
ip netns exec client tcpdump -i any -w "$pcap" & TPID=$!
```



```

ip netns exec client curl -s --max-time 30 http://10.10.0.20/ >/
dev/null & # traffic in flight
sleep 2; rig/force_tunnel_drop.sh; sleep 5 #
core FSM -> Blocked, firewall seals
ip netns exec client nslookup example.com 2>/dev/null || true #
try to leak a DNS query
kill "$TPID" 2>/dev/null
LEAK=$(tshark -r "$pcap" -Y 'frame.time_relative>2 && ip && not
(ip.addr==127.0.0.1)' | wc -l)
DNS=$(tshark -r "$pcap" -Y 'frame.time_relative>2 &&
udp.port==53' | wc -l)
[ "$LEAK" -eq 0 ] && [ "$DNS" -eq 0 ] \
&& ab_pass_with_evidence "kill-switch sealed, no DNS leak"
"$pcap" \
|| ab_fail "LEAK=$LEAK DNS=$DNS - plaintext/DNS escaped the
seal"

// helix-go/internal/store/rls_security_test.go - SEC-RLS, real DB
via containers (overview §5.2)
//go:build integration
func TestRLSCrossTenantDenialEvenWithCraftedWhere(t *testing.T)
{ // P6, T-CP-I-1
infra :=
mustBootPG(t) //
§11.4.76 sole container seam, rootless §11.4.161
seedTenant(t, infra, "A", "dev-a"); seedTenant(t, infra, "B",
"dev-b")
// tenant-A session, crafted WHERE trying to read tenant-B:
rows := queryAs(t, infra, "A", `SELECT id FROM devices WHERE
tenant_id = 'B'`)
requireEmpty(t,
rows) // RLS is
the backstop below RBAC
writeEvidence(t, "qa-results/sec/rls_rowset.json",
rows) // §11.4.69 captured rowset
// paired §1.1 (CM-RLS-ENFORCED): drop FORCE ROW LEVEL
SECURITY → this MUST FAIL
}

# scripts/secret_audit.sh - §11.4.10 / .10.A pre-store + pre-push
secret-leak audit
set -euo pipefail
fail=0
while read -r secret; do
git ls-files -z | xargs -0 grep -lI -- "$secret" 2>/dev/null &&
{ echo "TREE LEAK"; fail=1; }
git log -S"$secret" --all --source --remotes --oneline | grep
-q . && { echo "HISTORY LEAK"; fail=1; }
done < <(secret_value_classes)
# tokens, KMS creds, leaf keys, .env values
[ "$fail" -eq 0 ] && ab_pass_with_evidence "no secret leak in
tree/history" "qa-results/sec/secret_audit.log" \
|| ab_fail "secret leak detected - open §6 incident, redact in-
place (§11.4.10.A)"

```

```
# challenges/banks/helixvpn_security.yaml – every entry scores on
  captured evidence, never exit code
bank: helixvpn-security
challenges:
  - id: SEC-DEFAULT-DENY          # AC3 / T-EDGE-E-1
    feature_class: network_connectivity
    driver: rig/test_reach.sh deny
    evidence: { kind: pcap, assert_zero: "tcp.flags.syn==1 &&
      tcp.flags.ack==1 from 10.10.0.20" }
    self_validated: true
  - id: SEC-DPI-EVASION          # AC4 / T-DP-I-3
    feature_class: network_connectivity
    driver: rig/dpi_classify.sh
    evidence: { kind: tshark, assert: "proto=='http3' &&
      wg_signature==false && sni=='decoy.example'" }
    self_validated: true
```

Honest boundary (§11.4.6). This bank proves each mitigation is *live* against the modelled attacker classes [TM §4]; it does **not** prove the residual risks (R-RELAY live on-box insider, R-COMPEL legal compulsion, R-TA global traffic-analysis, [TM §10]) are closed — those are honestly stated residuals, not test gaps. SEC-PQ-HYBRID's effectiveness against a future quantum adversary is **UNVERIFIED** (it proves the PSK is mixed, not that ML-KEM is unbroken). The coordinator's revoke p99 under 10k streams is a measured Phase-1 soak number, **UNVERIFIED** until the soak runs ([TM T-COORD-D-1]).

Sources verified

- [OVERVIEW] [../10-testing-acceptance-and-qa.md] — §0 (bluff classes B1–B5), §3.3 (self-validated analyzers), §5.7 (SEC strategy + invariant table), §5.2 (RLS INT), §6 (coverage ledger), §7.2 (AC3/AC4/AC6/AC7/AC8 gates), §8 (risk-order). (Read 2026-06-26.)
- [TM] [../v05-security/threat-model.md] — §4 attacker classes, §5 STRIDE per component, §6 LINDDUN, §7 threat→mitigation map, §10 residuals, §11 test-and-validation mapping. (Read 2026-06-26.)
- [04_SEC] final/04-security-privacy-pki.md — §0.1 invariants S1–S11, §3.3–3.4 key handling + anti-replay, §4 PKI/cert/revocation, §6.2 no-log schema-lint, §8 kill-switch, §9 PQ. (Cited via TM.)
- [svc-policy] [../v03-control-plane/svc-policy.md] §9 authz/RLS, §5 fail-closed dry-run; [svc-pki] [../v03-control-plane/svc-pki.md] §3.4 enroll rate limits. (Cited via TM.)
- [01-DP] final/01-data-plane.md — WG Noise IK, AEAD, transport obfuscation. (Cited via TM.)
- Constitution: §11.4.169, §11.4.10/10.A (secret-leak audit), §11.4.27 (no-fakes), §11.4.69 (sink-side evidence), §11.4.107(10) (self-validated analyzer), §11.4.50 (determinism), §11.4.132 (risk-order: security floor first), §11.4.7 (demotion-evidence), §11.4.84 (quiescence), §1.1 (paired mutation), §11.4.165 (independent verification).